

Attacchi Avanzati in Assembly x64 (Canary e GOT)

NECKER

17 Novembre 2025

Contents

1	Analisi degli Esempi di Buffer Overflow Classico (Stack 1, 4, 5)	2
1.1	1. Stack 1.c: Obiettivo Semplice e Fisso	2
1.2	2. Stack 4.c: Attacco al Return Address (Problema del Byte Nullo)	3
1.3	3. Stack 5.c: Transizione a x64 (64-bit)	3
1.3.1	3. Stack 5.c: Iniezione di Shellcode e Ritorno allo Stack	4
2	Difesa Principale: Lo Stack Canary	4
3	Attacco Avanzato: Bypass del Canary con Return-to-GOT (Abo5.c)	5
3.1	L'Obiettivo: La Global Offset Table (GOT)	5
3.2	Meccanismo dell'Exploit in 3 Fasi	5
4	Attacchi Avanzati in Ambienti Protetti (NX bit)	5
4.1	RET2libc / ROP CHAIN: Esecuzione di Codice Esistente	6
4.2	Appunto	6
5	Esempio Pratico: PwnCollege Memory Errors – Level 1.1	7

1 Analisi degli Esempi di Buffer Overflow Classico (Stack 1, 4, 5)

Questi esempi illustrano come viene sfruttato un Buffer Overflow per manipolare variabili locali (il "cookie") anziché l'indirizzo di ritorno, a scopo didattico. La vulnerabilità in tutti i casi è l'uso della funzione `gets()`.

1.1 1. Stack 1.c: Obiettivo Semplice e Fisso

`stack1.c` dimostra il concetto base di Buffer Overflow per alterare una variabile locale.

```
int main()
{
    int cookie;
    char buf[80];

    printf("buf: %08x cookie: %08x\n", &buf, &cookie);
    gets(buf);

    if (cookie == 0x41424344)
        printf("you win!\n");
}
```

- **Vulnerabilità:** La funzione `gets(buf)` legge input senza limiti, permettendo di scrivere oltre i 80 byte di `buf`.
- **Obiettivo:** La variabile `cookie` è allocata nello Stack subito dopo `buf`.
- **Attacco:** L'aggressore fornisce una stringa di 80 byte + 4 byte (padding/allineamento) seguita dal valore target `0x41424344` (che corrisponde alla stringa "ABCD" in ASCII, endianness permettendo) per sovrascrivere `cookie` e vincere.
ATTENZIONE: devono essere scritti al contrario perché `int` è in little endian.
- **Output di debug:** L'uso di `%08x` suggerisce un'architettura a 32-bit (x86), dove gli indirizzi e gli interi sono a 4 byte.

1.2 2. Stack 4.c: Attacco al Return Address (Problema del Byte Nullo)

Sebbene il codice di prima miri alla sovrascrittura della variabile `cookie`, ora ci viene difficile poiché `cookie` contiene caratteri terminatori, per rompere `stack4.c`, dobbiamo infatti puntare a sovrascrivere l'indirizzo di ritorno (**RET**).

```
int main()
{
    int cookie;
    char buf[80];

    printf("buf: %08x cookie: %08x\n", &buf, &cookie);
    gets(buf);

    if (cookie == 0x000d0a00)
        printf("you win!\n");
}
```

L'Obiettivo dell'Attacco (Riscrivere il RET):

- **Bersaglio:** L'indirizzo di ritorno (**RET**) che si trova nello Stack Frame, dopo `buf` e `cookie`.
- **Azione:** L'aggressore deve inviare una stringa di 80 byte + Padding (per le variabili locali) seguita dal **nuovo Indirizzo di Ritorno** (l'indirizzo dello shellcode).

La Sfida Cruciale: La Funzione `gets()`

La funzione `gets()` legge l'input fino a quando non incontra un carattere di a capo (`\n`) o, più criticamente, un **Byte Nullo** (`\0`).

- **Possibile Problema:** L'indirizzo dello **shellcode** o di una funzione utile (come l'indirizzo di `system()`) contiene quasi sempre uno o più byte nulli (`0x00`) all'inizio o alla fine.
- **Il Fallimento:** Se l'aggressore tenta di includere un indirizzo contenente `0x00` nel payload di `gets()`, la funzione `gets()` **terminerà la lettura prematuramente** al primo byte nullo incontrato. L'indirizzo di ritorno non verrà scritto per intero, e l'attacco fallirà.
- **La Soluzione:** Per sfruttare `gets()`, l'aggressore deve trovare un modo per iniettare l'input *binario* che includa il byte nullo, spesso inviando il payload tramite un file o un pipe, e deve assicurarsi che l'indirizzo di salto scelto **non contenga byte nulli** (*null-byte free*) o che il byte nullo sia posizionato strategicamente per terminare la stringa dopo l'indirizzo e non al suo interno.

1.3 3. Stack 5.c: Transizione a x64 (64-bit)

`stack5.c` introduce il contesto di un'architettura a 64-bit, che complica ulteriormente l'allineamento e la dimensione delle variabili.

```
int main()
{
    int cookie;
    char buf[80];

    printf("buf: %16lx cookie: %16lx\n", &buf, &cookie);
    gets(buf);

    if (cookie == 0x000d0a00)
        printf("you loose!!\n");
}
```

- **Output di debug:** L'uso di `%16lx` suggerisce un'architettura **x64 (64-bit)**, dove i puntatori sono a 8 byte (16 cifre esadecimali).
- **Variabile di Controllo:** Nonostante l'architettura a 64-bit, la variabile `cookie` rimane di tipo `int` (solitamente 4 byte).

- **Implicazione dell'Overflow:** Lo spazio tra `buf[80]` e la variabile `cookie` è ora più complesso e probabilmente include un **padding** (spazio vuoto) per garantire che l'indirizzo di `cookie` sia allineato correttamente a 4 o 8 byte (come richiesto dalle convenzioni x64).
- **Attacco:** L'aggressore deve calcolare con precisione la lunghezza del padding (che può essere fino a 4 byte in più) oltre agli 80 byte di `buf` per colpire correttamente `cookie`.
ATTENZIONE: ora però se indoviniamo `cookie` perdiamo, quindi iniettiamo shell code:

1.3.1 3. Stack 5.c: Iniezione di Shellcode e Ritorno allo Stack

è possibile eseguire l'attacco classico di Buffer Overflow (se le difese moderne sono disattivate) iniettando un codice malevolo (*shellcode*) direttamente nello Stack e forzando il programma ad eseguirlo.

Presupposti per l'Attacco:

- **Stack Eseguitabile (NX bit disattivato):** La sezione dello Stack, dove viene iniettato il *shellcode*, deve consentire l'esecuzione del codice.
- **Canary Disattivato:** La protezione Stack Canary deve essere assente o inefficace per consentire la sovrascrittura del RET.

Fasi dell'Attacco:

1. **Calcolo dell'Indirizzo di Iniezione:** L'attaccante usa l'output di `printf (%16lx)` per determinare l'indirizzo esatto di `buf` (es. `ADDRbuf`).
2. **Costruzione del Payload:** Il payload (la stringa fornita a `gets()`) viene strutturato come segue:

$$\text{Payload} = \underbrace{\text{Shellcode}}_{\text{In binario}} \quad || \quad \underbrace{\text{Padding}}_{\text{Riempimento fino al RET}} \quad || \quad \underbrace{\text{ADDR}_{buf}}_{\text{Nuovo Indirizzo di Ritorno (dobbiamo saperlo esattamente)}}$$

3. **Iniezione:** Il *shellcode* viene iniettato nella parte iniziale del buffer `buf`. La lunghezza dello shellcode e del padding riempie tutto lo spazio tra l'inizio di `buf` e il **Return Address (RET)**.
4. **Dirottamento del RET:** Gli ultimi 8 byte del payload (su x64) sovrascrivono il RET con l'indirizzo di `buf` (`ADDRbuf`).
5. **Esecuzione:** Quando la funzione `main` termina, invece di eseguire l'istruzione RET per tornare al chiamante legittimo, il processore salta all'indirizzo iniettato, che è l'inizio di `buf` dove si trova lo *shellcode*. Lo shellcode viene quindi eseguito con successo.

2 Difesa Principale: Lo Stack Canary

Gli attacchi moderni sfruttano la separazione logica della memoria per bypassare le difese.

Elemento	Dove si Trova	Scopo
Return Address (RET)	Stack Frame	Indirizzo di ritorno della funzione.
Stack Canary	Stack Frame	Protezione del RET.
Global Offset Table (GOT)	Sezione Dati/BSS	Contiene gli indirizzi delle funzioni di libreria (<code>exit</code> , <code>printf</code>).

Table 1: Differenze tra Stack e Dati

- **Funzione:** Il Canary è un valore segreto posizionato nello Stack Frame tra le variabili locali (es. il buffer `buf`) e il Return Address (RET).
- **Meccanismo di Protezione:** Prima che la funzione termini, il programma verifica l'integrità del Canary.
- **Reazione al Fallimento:** Se il Canary è stato modificato (segno di un Buffer Overflow), il programma **non** esegue l'istruzione RET. Salta invece a una funzione di gestione degli errori che esegue `exit()` per terminare il programma in sicurezza.

3 Attacco Avanzato: Bypass del Canary con Return-to-GOT (Abo5.c)

```
int main(int argv, char **argc) {
    char *pbuf = malloc(strlen(argc[2]) + 1);
    char buf[256];

    strcpy(buf, argc[1]);

    for (; *pbuf++ = *(argc[2]++); );
    exit(1);
}
```

L'attacco si concentra sull'aggiramento della difesa del Canary, prendendo di mira la **GOT** quando il Canary costringe il programma ad uscire.

3.1 L'Obiettivo: La Global Offset Table (GOT)

- La **GOT** è l'elenco di puntatori che il programma usa per chiamare le funzioni in librerie esterne (come `exit`).
- **L'Attacco:** Riscrivere l'indirizzo della funzione `exit` all'interno della GOT con l'indirizzo dello shellcode dell'aggressore.

3.2 Meccanismo dell'Exploit in 3 Fasi

Fase 1: Ottenere la Scrittura Arbitraria (Dirottamento del Puntatore)

- **Vulnerabilità:** `strcpy(buf, argv[1])`
- **Azione:** L'aggressore fornisce una stringa di `argv[1]` che è abbastanza lunga da uscire dal `buf[256]` e sovrascrivere il puntatore locale `char *pbuf` nello Stack.
- **Risultato:** `pbuf` viene forzato a puntare all'indirizzo della **GOT** di `exit` (ATTENZIONE: l'indirizzo della GOT deve essere noto).

Fase 2: Scrivere il Payload (Shellcode)

- **Vulnerabilità:** `for (; *pbuf++ = *argv[2]; );`
- **Azione:** L'aggressore usa `argv[2]` per fornire il **codice dello shellcode** o l'indirizzo della funzione di attacco. Il ciclo copia questo contenuto nell'indirizzo puntato da `pbuf`.
- **Risultato:** L'ingresso della **GOT** di `exit` viene sovrascritto con l'indirizzo dello shellcode.

Fase 3: Esecuzione del Codice Malevolo

- **Innesco:** Il programma arriva alla riga `exit(1);` (o il Canary fallisce e chiama `exit()`).
- **Esecuzione:** L'istruzione `CALL exit` consulta la **GOT**. Legge l'indirizzo dello shellcode scritto dall'aggressore.
- **Conclusione:** Il flusso di esecuzione salta allo shellcode, bypassando la protezione dello Stack Canary.

4 Attacchi Avanzati in Ambienti Protetti (NX bit)

Le tecniche precedenti (iniezione di shellcode nello Stack) falliscono se lo Stack è marcato come **non eseguibile** (grazie al *No-eXecute bit*, o NX bit). Il prossimo esempio mostra come aggirare questo problema.

ATTENZIONE: nel prossimo esempio non ci deve essere il canary attivo.

4.1 RET2libc / ROP CHAIN: Esecuzione di Codice Esistente

Se lo Stack non è eseguibile, l'aggressore non può eseguire il codice iniettato. La soluzione è **riutilizzare** codice che è già in memoria e che è garantito essere eseguibile, come le funzioni della libreria standard `libc` (da qui il nome RET2libc, Return-to-libc). Si chiama chain perché usa codice già presente ma sparso e lo collega a catena per fare ciò che ci serve

```
int gadget()
{
    asm(
        "pop %rdi\n"
        "ret");
}

int main()
{
    int cookie;
    char buf[80];

    printf("puts(): %16lx\n", &puts);
    printf("gadget(): %16lx\n", &gadget);
    printf("buf: %16lx cookie: %16lx\n", &buf, &cookie);
    gets(buf);

    if (cookie == 0x000d0a00)
        printf("you loose!\n");
}
```

- **Obiettivo:** Chiamare una funzione esistente (*gadget* o funzione di libreria, es. `system()`) usando l'overflow per sovrascrivere il **Return Address (RET)**.
- **Problema x64:** Per chiamare una funzione (es. `puts()`), l'architettura x64 (System V ABI) richiede che il primo argomento (es. l'indirizzo della stringa da stampare) sia caricato nel **registro RDI**. Lo Stack Overflow consente di controllare solo il RET, non i registri.
- **Soluzione: Gadget POP RDI (ROP Chain):** L'attaccante cerca nella memoria del programma una breve sequenza di istruzioni Assembly detta **Gadget** (es. `"pop rdi; ret"`). Questo gadget, quando viene eseguito:
 1. Estrae il valore successivo dallo Stack e lo carica nel registro RDI.
 2. Esegue RET, che salta all'indirizzo successivo sullo Stack.

Struttura della ROP Chain (Stack Dirottato):

La catena di esecuzione viene costruita interamente nello Stack (non eseguibile):

1. Il **RET** originale viene sovrascritto con l'indirizzo del **Gadget** (`pop rdi; ret`).
2. Subito dopo, viene posizionato l'**Argomento** per la funzione (es. il puntatore alla stringa `"You Win!\0"`).
3. Subito dopo, viene posizionato l'**Indirizzo** della funzione `puts()` (o `system()`).

Quando il programma esegue il RET, esegue il Gadget, che prepara RDI e salta alla funzione, ottenendo così l'esecuzione con i parametri desiderati.

4.2 Appunto

ricordiamo che lo **Stack** e il **.Text** (sezione dove viene scritto il codice), non sono consapevoli l'uno dell'altro e confidano sul fatto di essere allineati in quello che fanno, questo esempio fa in modo di disallineare lo **Stack** in modo da far prelevare quello che vogliamo noi come se fosse il **RET**.

STACK DOPO LA SOVRASCRITTURA DA OVERFLOW:

Indirizzo Relativo	Contenuto (Payload Iniettato)
Old RBP	Indirizzo di <code>main</code> precedente (8 byte)
Vecchio RET	<code>&gadget()</code>
RET + 8	Indirizzo della Stringa (<code>&"You Win!\0"</code>)
RET + 16	<code>&puts()</code>
buf	80 byte da riempire - quelli usati
stack non eseguibile	
stack non eseguibile	
String	<code>&"You Win!\0"</code>

5 Esempio Pratico: PwnCollege Memory Errors – Level 1.1

Questo esempio illustra un exploit che non mira al flusso di controllo (*RET* o *GOT*), ma alla **logica** del programma sovrascrivendo una variabile di controllo.

```
; [RBP - 0x90] contiene l'indirizzo del buffer (RBP-0x80)
; questo lo sappiamo da queste 2 istruzioni precedenti:
lea rax, [rbp - 0x80]
mov QWORD PTR [rbp - 0x90], rax ;salva in 0x90, 0x80

; Carica l'indirizzo del buffer in RAX
mov rax, QWORD PTR [rbp-0x90]

; Imposta EDI (RDI) = 0 (dice a read di prendere da stdin
;(poi lo mettera' in cio' che c'e' scritto in rsi))
mov edi, 0x0

; Imposta RSI (Argomento 2) = Indirizzo del buffer (RBP-0x80)
mov rsi, rax

; Carica la dimensione della lettura in RDX (Argomento 3)
; Il valore qui e' eccessivo, causando l'overflow
mov rdx, QWORD PTR [rbp-0x98]

; Esegue la funzione di lettura. L'overflow avviene qui.
call 0x11a0 <read@plt>
```

Obiettivo del Challenge:

- Trovare un modo per chiamare il metodo `win()`.

Meccanismo di Vittoria:

- Il codice disassemblato mostra che il programma esegue un controllo: se il valore a cui punta la variabile locale `[RBP - 0x88]` è diverso da zero (`!= 0`), la funzione `win()` viene chiamata.

La Vulnerabilità Sfruttabile (Scrittura Arbitraria Indiretta):

- La funzione `read()` (che è non sicura, simile a `gets`) legge i dati in un buffer la cui posizione di inizio è memorizzata in `[RBP - 0x90]` (che, a sua volta, contiene l'indirizzo `RBP - 0x80`).
- L'overflow si verifica quando l'input in `read()` supera il buffer.
- **Strategia di Attacco:** L'attaccante sfrutta l'overflow per:

1. **Sovrascrivere il Puntatore:** La funzione `read()` è chiamata per scrivere nel buffer. Il primo overflow è abbastanza grande da sovrascrivere l'indirizzo memorizzato in `[RBP - 0x88]` (il puntatore alla variabile di controllo).

2. **Sovrascrivere il Valore:** L'input viene fornito in modo che, in una fase successiva, venga scritto un valore diverso da zero (*e.g.*, 1) all'indirizzo che `[RBP - 0x88]` ora punta.
- **Conclusione:** Poiché l'utente può controllare la dimensione dell'input, è possibile sovrascrivere il valore controllato dalla condizione `if` e forzare l'esecuzione della funzione `win()`.